

ResuMatch Quick & Easy Interview Guide

Top 50 Concise & High-Impact Questions and Answers

React 19 ♦ Vite 7 ♦ Node.js ♦ Express 5 ♦ MongoDB ♦ Gemini AI

50 Core Q&As

1. Project Overview & Architecture

Q1: What is ResuMatch in simple words?

ResuMatch is a web application that helps job seekers prepare for interviews.

- ♦ The user uploads a PDF resume and pastes a job description.
- ♦ The AI compares both, calculates a match score, highlights missing skills, and creates a step-by-step interview preparation plan.

Q2: What is the tech stack of ResuMatch?

- ♦ Frontend: React 19, Vite 7, React Router 7, Sass, Axios.
- ♦ Backend: Node.js, Express 5, Multer, pdf-parse, Puppeteer.
- ♦ Database: MongoDB Atlas with Mongoose ODM.
- ♦ AI & Auth: Google Gemini API (@google/genai) and JWT in HTTP-Only cookies.

Q3: Why separates Frontend and Backend (Decoupled Architecture)?

- ♦ Independent Scaling: Frontend and Backend can be updated and hosted separately.
- ♦ User Experience: React handles page routing instantly without reloading the entire page.
- ♦ Reusability: The backend REST API can serve mobile apps or extensions in the future.

Q4: What is the step-by-step flow when a user uploads a resume?

1. User selects a PDF file in the React frontend.
2. Axios sends a multipart form request to Express backend.
3. Multer receives the file buffer in memory.
4. pdf-parse extracts the raw text from the buffer.
5. Gemini AI processes text + job description and returns structured JSON.
6. MongoDB saves the report and React displays it to the user.

Q5: What design patterns are used in this project?

- ♦ Feature-based Directory Pattern (Grouping files by auth/interview feature).
- ♦ Controller-Service-Model Pattern on the backend.
- ♦ Custom Hook Pattern (useAuth, useInterview) for clean React code.

2. Frontend (React 19 & Vite 7)

Q6: Why did you use Vite instead of Create React App?

Vite is significantly faster. It starts the local dev server instantly using native ES Modules and esbuild, whereas Create React App (CRA) uses heavy Webpack bundling and is now deprecated.

Q7: How does React Context API work in your project?

Context API provides global state without passing props through every component.

- ♦ AuthContext holds user data and login status.
- ♦ InterviewContext holds active reports and loading states.

Q8: How do Protected Routes work?

We built a wrapper component .

- ◆ It checks if user is logged in.
- ◆ If logged in → renders the page.
- ◆ If not logged in → automatically redirects to /login using React Router.

Q9: How do you validate files on the frontend before sending to the server?

Inside the file selection handler, we verify:

- 1) File format must be PDF (file.type === "application/pdf").
 - 2) File size must be under 3MB (file.size <= 3MB).
- If invalid, we show an error immediately without calling the backend.

Q10: What are Custom Hooks and why use them?

Custom hooks (like useAuth and useInterview) extract complex logic away from UI components. Components simply call const { generateReport, loading } = useInterview() without worrying about Axios promises.

Q11: How does Axios handle login session cookies?

Axios is configured with withCredentials: true. This tells the browser to automatically include HTTP-Only authentication cookies in every API request.

Q12: How do you handle loading and error states in UI?

- ◆ Loading: Display a centered loading spinner and descriptive message while waiting for API responses.
- ◆ Error: Render clear, red banner alerts when an API request fails.

Q13: Why choose Sass over plain CSS or Tailwind?

Sass allows nested styling rules, variables, and mixins for clean modular code without cluttering JSX elements with dozens of utility classnames.

Q14: How is mobile responsiveness handled?

Using CSS Grid/Flexbox layouts and media queries (@media max-width: 768px). Cards stack vertically and navigation adjusts for smaller screens.

Q15: How does PDF downloading work on the frontend?

Axios receives binary PDF data (blob). We generate a temporary download URL using window.URL.createObjectURL(blob), trigger an automatic click on a hidden link, and remove it.

3. Backend (Node.js & Express 5)

Q16: Why Express 5 for the backend?

Express 5 provides modern improvements: automatic handling of rejected promises in async route handlers, faster routing, and updated security standards.

Q17: What is Multer and how is it used here?

Multer is middleware for handling file uploads. We configured it with multer.memoryStorage(), which keeps the uploaded PDF buffer directly in RAM (memory) for fast text extraction.

Q18: Why memory storage instead of saving files to disk?

We only need the text inside the PDF to send to Gemini AI. Memory storage is faster, avoids disk I/O, and removes the need to write cleanup scripts for temporary files.

Q19: How does pdf-parse read resume text?

pdf-parse takes the binary Buffer from RAM, parses PDF text elements across all pages, and returns the raw text string ready for AI analysis.

Q20: How do you prevent the backend server from crashing on errors?

Every async controller function is wrapped in a try...catch block. If an error occurs, it is logged to the console and a clean 500 error response is sent back to the user.

Q21: How is CORS configured on the backend?

Using the cors package configured with an array of allowed origins (http://localhost:5173, 5174, 5175) and credentials: true.

Q22: What is the middleware execution sequence?

Incoming Request → express.json() → cookieParser() → cors() → authUser middleware → Controller action.

Q23: How does cookie-parser work?

It parses the raw Cookie string from HTTP request headers and attaches it as a clean JavaScript object to req.cookies.

Q24: How does dotenv manage environment secrets?

dotenv.config() loads variables from .env into process.env at server startup so credentials are never hardcoded.

Q25: What HTTP status codes are used?

- 💎 200: Success
- 💎 201: Resource Created
- 💎 400: Bad Request / Validation Failure
- 💎 401: Unauthorized
- 💎 404: Not Found
- 💎 503: Service Unavailable

4. Database (MongoDB & Mongoose)

Q26: What collections exist in MongoDB?

1. User: Credentials & profiles
2. InterviewReport: AI generated interview strategies
3. Blacklist: Invalidated JWT tokens for logout

Q27: How is the User Model structured?

Stores username, email, and password. Username and email have unique: true indexes to prevent duplicate accounts.

Q28: How is InterviewReport document structured?

Stores user ObjectId, raw resume text, job description, match score, arrays for technical questions, behavioral questions, skill gaps, and preparation plan.

Q29: How does Mongoose connect to MongoDB Atlas?

Using mongoose.connect(process.env.MONGO_URI) with an encrypted TLS/SSL connection string stored in .env.

Q30: How are database queries optimized?

- 💎 Indexing the user field for fast lookups.
- 💎 Using Mongoose projections (.select("-resume -jobDescription")) when fetching list views to reduce payload size.

Q31: What is a TTL (Time-To-Live) Index in MongoDB?

TTL indexes automatically delete documents after a set time. The Blacklist collection uses expires: "1d" so expired logout tokens auto-delete after 24 hours.

Q32: Why use Mongoose ODM over raw MongoDB driver?

Mongoose provides schema validation, middleware hooks, strict typing, and clean chainable query methods.

5. AI Integration & Gemini API

Q33: How is Gemini AI integrated into Node.js?

Using the official @google/genai SDK. We instantiate GoogleGenAI({ apiKey }) and call ai.models.generateContent().

Q34: How do you force Gemini to return JSON instead of text?

We define a Zod schema, convert it using zod-to-json-schema, and pass it in config.responseSchema with responseMimeType: "application/json".

Q35: What Gemini model is used?

gemini-1.5-flash 💎 optimized for speed, low token cost, and reliable structured JSON output.

Q36: How does ATS Resume PDF generation work with Puppeteer?

1. Gemini generates ATS-optimized HTML string.
2. Puppeteer opens a headless browser tab.
3. Sets HTML content and renders an A4 PDF buffer.
4. Streams PDF buffer directly to user.

Q37: What happens if the Gemini API Key is missing?

AI client creation is deferred to request time inside getAiClient(). If missing, the backend returns a clean 503 error without crashing the server.

Q38: How do you prevent prompt injection or bad inputs?

Client and server validate character limits (5000 max), check non-empty text, and enforce PDF mime types.

6. Security & Authentication

Q39: How is authentication implemented securely?

1. Passwords hashed with bcryptjs (salt round 10).
2. Login generates a signed JWT token.
3. JWT sent via HTTP-Only cookie.
4. Auth middleware verifies cookie on protected endpoints.

Q40: Why HTTP-Only Cookies over LocalStorage?

LocalStorage is vulnerable to XSS attacks (malicious scripts can read tokens). HTTP-Only cookies cannot be accessed via JavaScript.

Q41: How does password hashing work?

bcrypt.hash(password, 10) adds a random salt and hashes the password multiple times so original plaintext cannot be recovered.

Q42: How does Token Blacklisting work on logout?

When user logs out, the backend saves the JWT token into the MongoDB Blacklist collection and clears the cookie. Subsequent requests with that token are rejected.

Q43: What is JWT verification in middleware?

auth.middleware extracts cookie, checks blacklist, and verifies signature using `jwt.verify(token, JWT_SECRET)`. If valid, attaches user payload to `req.user`.

7. Troubleshooting & System Design

Q44: How did you fix the ERR_CONNECTION_REFUSED error?

Node v24 on Windows bound Vite to IPv6 (:::1). Browsers connecting via IPv4 127.0.0.1 failed. Added server: { host: "0.0.0.0", port: 5173 } in vite.config.js to bind to all interfaces.

Q45: How did you fix CORS issues when Vite changed ports?

Updated backend `app.use(cors({ origin: ["http://localhost:5173", "http://localhost:5174", "http://localhost:5175"] })))` to accept requests from common dev ports.

Q46: How would you scale this app to 100k daily users?

- 1) Run stateless Express instances behind NGINX load balancer.
- 2) Use Redis for token blacklisting.
- 3) Move PDF generation & AI tasks to background BullMQ worker queues.

Q47: How would you handle large file uploads in production?

Stream files directly to AWS S3 using presigned URLs or Multer-S3, avoiding sending heavy binary payloads through Node server.

Q48: How do you handle environment variables in production?

Store secrets in host platform environment managers (Render/Vercel/AWS Parameter Store) and never commit `.env` to Git.

Q49: What is the benefit of Zod schema validation?

Zod provides compile-time and runtime type safety, parsing raw JSON into validated TypeScript/JS objects.

Q50: What would you add to ResuMatch in the future?

- 1) Real-time AI mock voice interviews using WebRTC.
- 2) Live ATS score gauge.
- 3) Application tracking kanban board.